Contents

Speaker notes: Damian (slides 13–21) **1**
Slide 13 — Second workload: file-tools . . . 1
Slide 14 — File-tools: structure . . . 1
Slide 15 — File-tools: light request paths . . . 2
Slide 16 — File-tools: image-heavy paths . . . 2
Slide 17 — File-tools: cold start snapshot . . . 2
Slide 18 — What the two workloads taught us . . . 2
Slide 19 — Recommendations . . . 3
Slide 20 — Demo path . . . 3
Slide 21 — Final conclusions . . . 3
Quick answers if someone asks . . . 4

Speaker notes: Damian (slides 13–21)

About **5–7 minutes**. Pick up right after Miłosz on slide 13.

People have heard the Wasm versus Docker setup, but a one-line reminder is fine if you sense confusion. Speak like you're walking a colleague through the results, not reading a spec.

Slide 13 — Second workload: file-tools

Thanks, Miłosz.

So far you've seen movie-search — mostly text, mostly in memory. We added a **second** service on purpose, because one application shape isn't enough to generalize from.

file-tools is a small API that does the kind of work libraries usually do: validate JSON, convert between JSON and CSV, read image metadata, grayscale an image, resize an image. Nothing exotic, but more than a hello-world.

Again we built it twice with the same behavior. Spin runs as a Wasm component on port **3000**. Docker runs as a normal Rust server in a container on **8081**. The endpoints listed on the slide are the same on both sides.

Slide 14 — File-tools: structure

The picture should feel familiar after Miłosz's architecture slide.

In the middle is a shared contract — what each route must do. Spin and Docker are two implementations. Underneath, the same benchmark harness writes results to CSV files in `bench-results`.

We split the routes mentally into two buckets: **light JSON work**, and **image-heavy work**. We expected different winners in those buckets, and that's exactly what the next slides show. This slide is just the map before the numbers.

Slide 15 — File-tools: light request paths

Here we're at concurrency fifty — meaning fifty clients hitting the service at the same time — and we're looking at throughput, requests per second.

For the light routes — health, JSON validation, JSON to CSV, CSV to JSON — Spin is ahead on every bar in this chart. Roughly twenty to fifty percent more throughput than Docker in this run, and tail latency was similar or a bit better for Spin.

The way I read that: when the job is mostly parsing, routing, and small transformations inside a request handler, the Wasm path looked solid here. It backs up what Miłosz showed on search — containers aren't automatically faster for everything.

Slide 16 — File-tools: image-heavy paths

Now the picture flips.

On metadata, grayscale, and especially resize, Docker is much faster. For resize at concurrency fifty, we saw on the order of six hundred requests per second on Docker versus about eighty on Spin, and the slow requests on Spin could approach around eight hundred milliseconds — close to a second for the unlucky tail.

Intuitively, image work wants mature native libraries that have been tuned for years on normal Linux binaries. Wasm can still do the work, but when CPU and buffers dominate, the container/native path had a clear edge in our setup.

So slide fifteen and sixteen belong together: light JSON favored Spin; heavy images favored Docker. **Workload shape** matters more than the label on the box.

Slide 17 — File-tools: cold start snapshot

Quick cold-start snapshot for file-tools only: Docker around three hundred milliseconds median, Spin around five hundred sixty. That's only five runs each, so treat it as a hint, not the main event — Miłosz's movie-search cold-start study had more repetitions.

It fits the story — Spin took longer to be ready here — but I wouldn't headline the talk with this slide alone.

Slide 18 — What the two workloads taught us

If I step back from the charts, a pattern shows up across both services.

Spin and Wasm looked strongest when the handler is small, tied to a single request, and the work is things like parsing, routing, and light transforms — and when you care about sandboxing and portability more than plugging into every native library on the planet.

Docker and OCI looked strongest when native libraries and CPU-heavy code drive the cost — images, databases, memory-mapped storage — or when your team already lives in normal container ops and wants familiar tooling.

Neither column is “always choose this.” They’re “start here when your problem looks like this.”

Slide 19 — Recommendations

So what would we actually recommend?

Consider Spin or Wasm for edge handlers, plugins, untrusted extensions — code you need to isolate — small adapters, jobs where you want a tight permission list.

Stay on Docker for storage-heavy services, real databases, media pipelines, messy dependencies, and teams that already run Kubernetes or Docker with established playbooks.

And whatever you choose, benchmark your **own** endpoints: cold and warm, cheap routes and real user routes, averages **and** tail latency, memory measured as consistently as you can, and correctness **while** you’re under load — not only on a single manual curl.

Miłosz already showed that “Wasm is lighter” isn’t a safe assumption without measuring. Same idea here: measure first, then decide.

Slide 20 — Demo path

If we demo live, we keep it short — the full benchmark already ran offline.

For movie-search we’d bring up Spin on 8080 and Docker on 8081, run the parity script, maybe search for `space` in the UI and then try enhanced mode with a deliberate typo like `spce`. For file-tools we’d hit a JSON validation route and an image resize on both ports.

If something doesn’t start on stage, we have a fallback line ready: the full benchmark is already done; live demo proves the services work and match, and the saved CSVs and plots in `results` carry the performance story. We do **not** run a full load test in front of you — that stays in the artifacts.

Slide 21 — Final conclusions

To close: we ran equivalent microservices on Spin and on Docker, and we checked they behaved the same before we believed any speed numbers.

Wasm is practical for some serverless-style handlers — especially small, isolated request paths. Docker is still the safer default when you need heavy native code, storage, or standard operations.

The slogan “Wasm beats containers on memory” was too simple for our data. **How** you measure and **what** endpoint you measure mattered as much as the technology label.

The bottom line on the slide is the one I’d leave you with: choose your isolation substrate after you measure real endpoint behavior. Same code first. Measurements second. Recommendations last.

Thanks — happy to take questions.

Quick answers if someone asks

Why two workloads? So we don't overfit to one app — search versus library-style JSON and images.

Is Spin the same as Wasm? Spin is how we built and ran the service; wasmtime runs the Wasm; Docker runs a native process in a container image.

Can we trust the memory numbers? They're useful directionally, but Docker and Spin don't report memory the same way — Miłosz covered that on slide 9.